

Manual de Usuario - Golampi Compiler

1. Introducción

Golampi Compiler es una potente herramienta académica diseñada para compilar código fuente escrito en el lenguaje **Golampi** (inspirado en Go) hacia código ensamblador **ARM64** (AArch64). La herramienta ofrece una interfaz web intuitiva para el desarrollo, depuración y visualización de reportes técnicos.

2. Instalación y Configuración Paso a Paso

2.1 Requisitos del Sistema

- **Sistema Operativo:** Linux (Recomendado), macOS o Windows.
- **PHP:** Versión 8.0 o superior instalada.
- **Ensamblador (Opcional para ejecución):** aarch64-linux-gnu-as y ld.
- **Emulador (Opcional para ejecución):** qemu-aarch64.

2.2 Pasos para Instalar

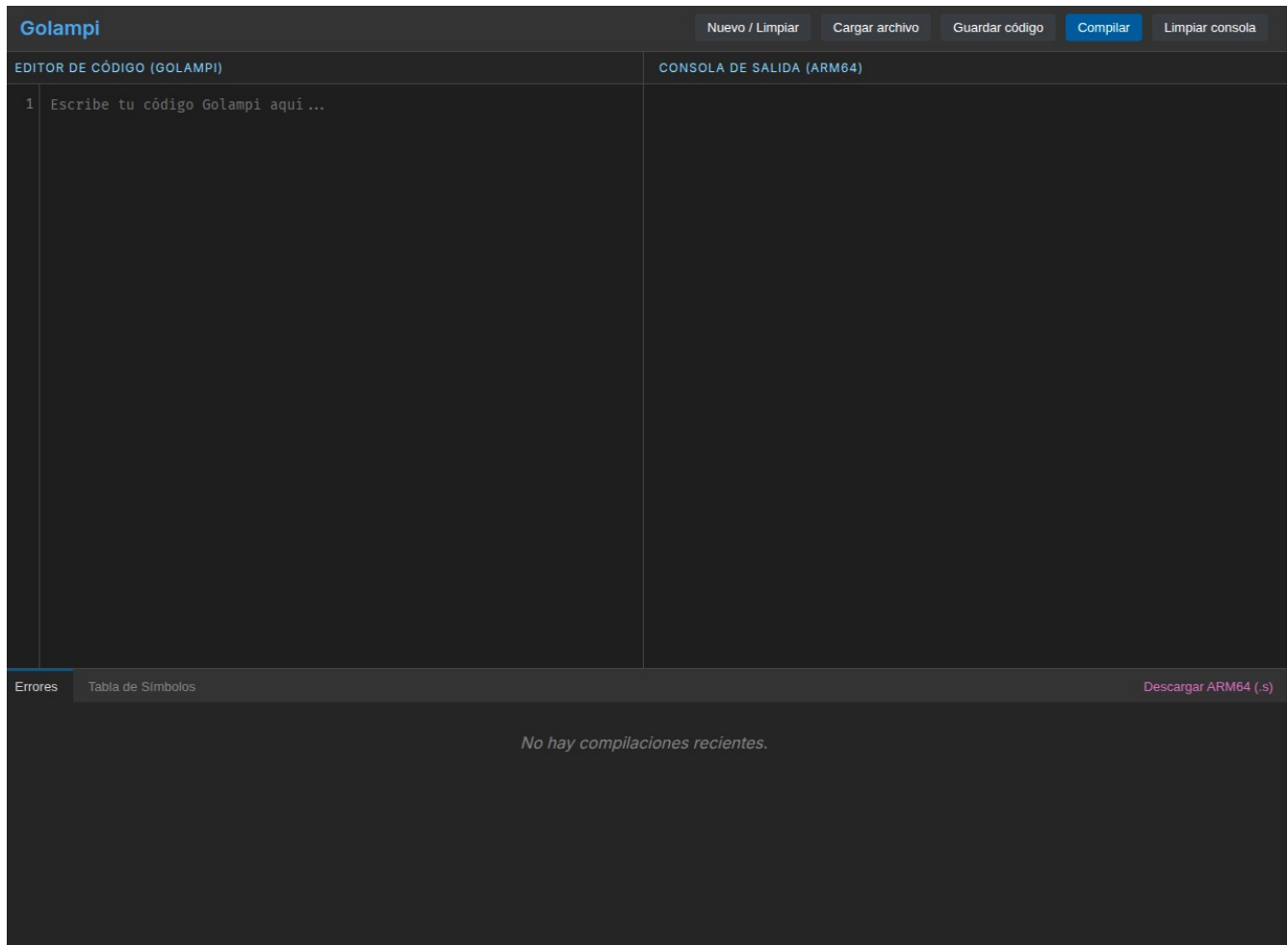
1. **Descargar el proyecto:** Extrae los archivos en una carpeta local de tu preferencia.
2. **Preparar Dependencias:** Asegúrate de que el archivo `antlr4.jar` se encuentre en la raíz del proyecto si planeas regenerar la gramática. El proyecto ya incluye las clases generadas en `src/generated`.
3. **Iniciar Servidor Local:** Abre una terminal en la carpeta raíz del proyecto y ejecuta:

```
php -S localhost:8000
```

3. Guía de Uso de la Herramienta

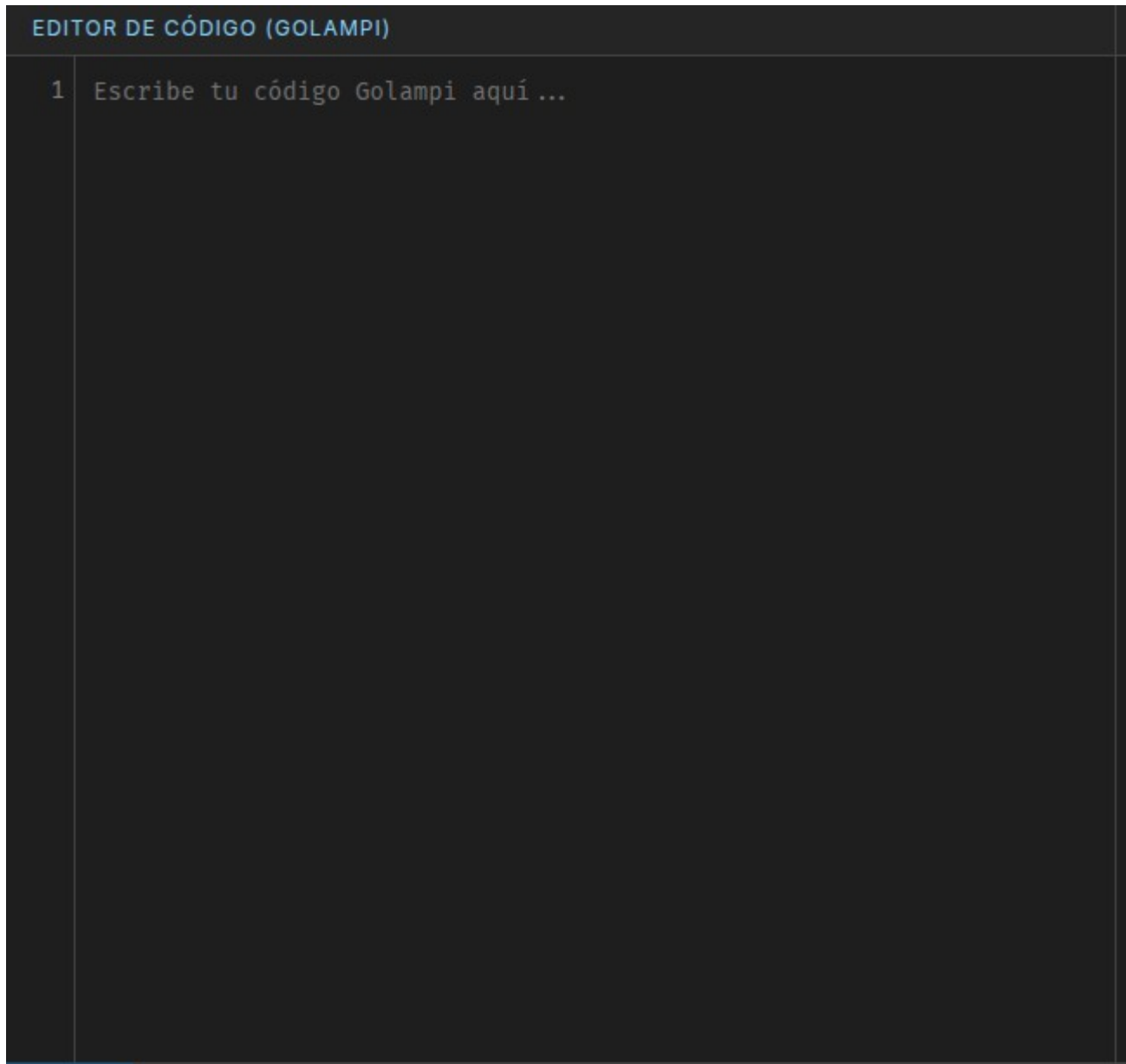
3.1 Acceso a la Interfaz

Abre tu navegador web y dirígete a `http://localhost:8000/frontend/index.html`. Se te presentará el entorno de desarrollo integrado (IDE).



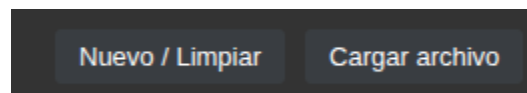
3.2 Crear y Editar Código

1.**Editor de Código:** Utiliza el panel izquierdo para escribir tu código. El editor cuenta con numeración de líneas.



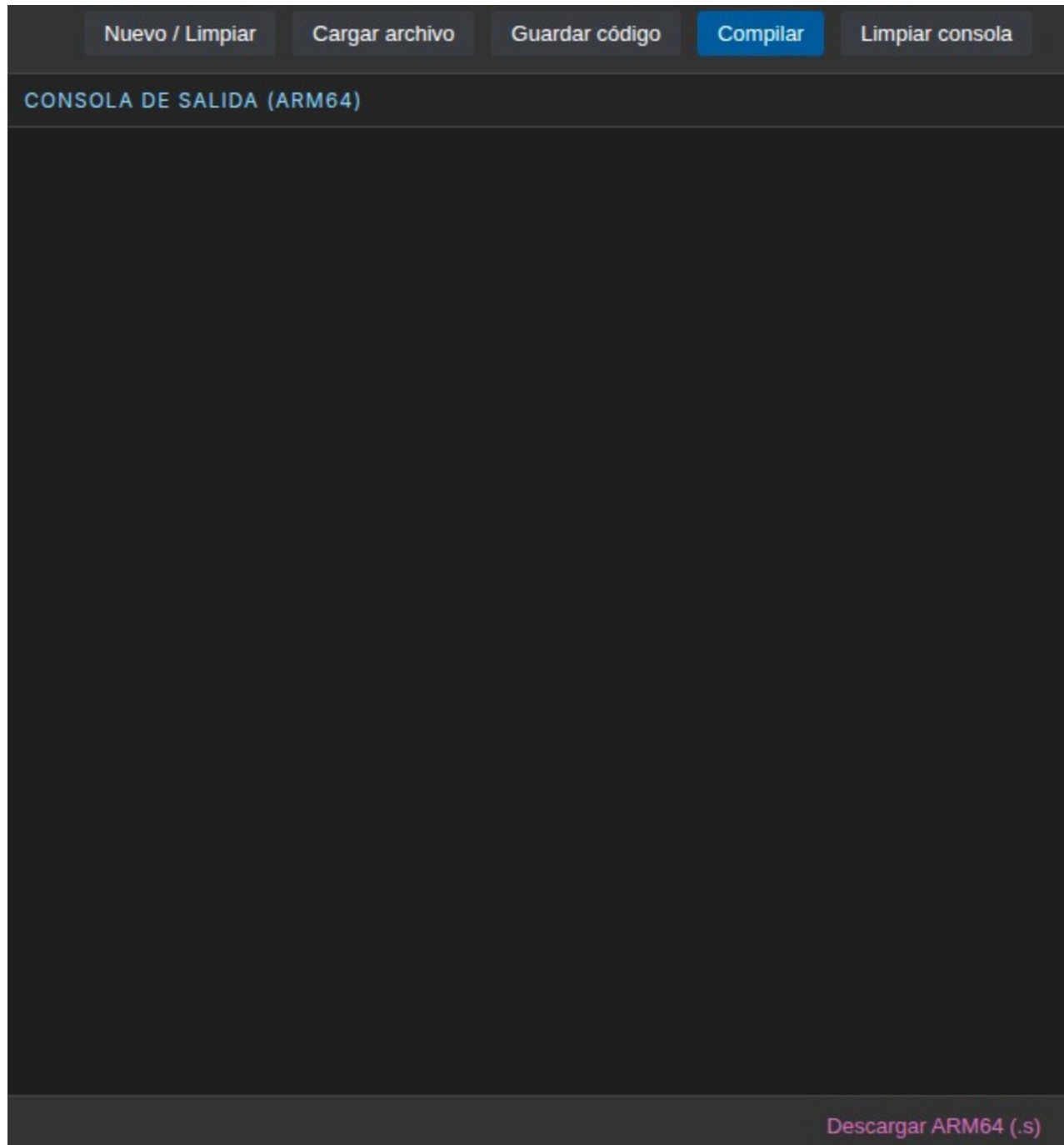
2.**Limpiar Editor:** El botón "**Nuevo**" borra todo el contenido actual para iniciar un nuevo proyecto.

3.**Cargar Archivo:** El botón "**Cargar Archivo**" permite importar archivos `.go` o `.gl`.



3.3 Compilación y Ejecución

- 1.**Botón Compilar:** Haz clic en el botón con el ícono de rayo (o texto "Compilar").
- 2.**Consola de Salida:** Si la compilación es exitosa, el panel derecho mostrará el código **ARM64** generado.
- 3.**Descarga:** Podrás descargar el archivo .S resultante para su ensamblaje manual.



4. Interpretación de Reportes

4.1 Reporte de Errores

Si tu código contiene errores (Léxicos, Sintácticos o Semánticos), se activará automáticamente la pestaña "**Errores**" en el panel inferior.

[!NOTE] [INSERTAR AQUÍ CAPTURA DE PANTALLA: TABLA DE ERRORES]

- Léxicos:** Caracteres no reconocidos.

Errores	Tabla de Símbolos				Descargar ARM64 (.s)
#	Tipo	Descripción	Línea	Columna	
1	Léxico	token recognition error at: '@'	12	0	

- Sintácticos:** Errores en la estructura del código (ej. falta un `;` o un `}`).

Errores	Tabla de Símbolos				Descargar ARM64 (.s)
#	Tipo	Descripción	Línea	Columna	
1	Sintáctico	no viable alternative at input 'varvarBoolboo'	22	13	

- Semánticos:** Errores de lógica o tipos (ej. sumar un `string` con un `int`).

4.2 Tabla de Símbolos

Accede a la pestaña "**Tabla de Símbolos**" para visualizar cómo el compilador ha registrado tus variables, funciones y constantes.

Errores	Tabla de Símbolos				Descargar ARM64 (.s)
Identificador	Tipo	Ámbito	Valor	Línea	Columna
main	func	global	Function	13	0
varInt	int32	local	Local (offset -48)	20	1
varFloat	float32	local	Local (offset -56)	21	1
varBool	bool	local	Local (offset -64)	22	1
varRune	rune	local	Local (offset -72)	23	1
varString	string	local	Local (offset -80)	24	1

En esta tabla verás:

- Nombre:** Identificador de la variable.
- Tipo:** `int32`, `float32`, `bool`, `string`, etc.
- Ámbito:** Global o el nombre de la función donde se declaró.
- Valor/Dirección:** Valor inicial o desplazamiento en la pila (offset).

5. Solución de Problemas Comunes

- El servidor no inicia:** Verifica que el puerto 8000 no esté ocupado por otra aplicación.
- Error de ANTLR:** Asegúrate de tener instalado Java si necesitas regenerar el Parser.
- Código no genera ASM:** Revisa la pestaña de Errores Semánticos; es posible que haya una incompatibilidad de tipos en tu código.